

SPM Project Report

Partitioned Hash Join with Duplicates
Module 2: From Partition Mapping to Full Join

Hamas Ali - 726267

1 Introduction

This report presents the implementation and evaluation of a partitioned hash join with duplicates for SPM Module 2. In Module 1, the main task was to design an efficient mapping from keys to partition identifiers. In this module, that mapping is integrated into the complete join pipeline, first in a sequential reference implementation and then in a parallel implementation using C++ threads.

The algorithm follows a fixed structure. Both input relations are partitioned using the same hash-based mapping, and records that belong to the same partition are grouped into contiguous memory regions. Once partitioning is complete, the join is performed independently on corresponding partitions. Since duplicates are allowed, the contribution of a matching key is determined by its multiplicity in relation R .

The sequential implementation defines the expected behavior of the algorithm and serves both as the correctness reference and as the baseline for performance evaluation. The parallel implementation preserves the same overall structure and is designed to study where parallelism is effective and where it is not. All experiments reported here were executed on node09 of the SPM cluster, and the raw outputs of the experiments are available in the `reports/` folder of the project.

2 Sequential Implementation

The sequential implementation follows the complete partitioned hash join pipeline and is intentionally kept as the reference version of the algorithm. Its purpose is not only to produce the correct output, but also to provide the baseline against which the parallel version is evaluated.

Hash Function and Partition Mapping

The partition mapping function used in Module 2 is the same one developed in Module 1. It starts from a 64-bit key and first folds it into 32 bits by XORing the lower and upper halves of the key. This folding step allows both halves of the original key to influence the final partition identifier instead of relying only on the lower bits. The resulting 32-bit value is then multiplied by Knuth's multiplicative constant, which improves the distribution of keys across partitions. Finally, the partition identifier is obtained using a shift-and-mask operation, exploiting the fact that the number of partitions P is restricted to powers of two.

This choice is appropriate for the project because it is simple, branch-free, and efficient, while still providing a better distribution than a trivial mapping such as `key & (P-1)`. The same mapping is used in both the sequential and the parallel implementations, ensuring a fair correctness and performance comparison.

Partitioning Phase

The partitioning of a relation is performed in three steps. First, a **histogram** is computed to count how many records map to each partition. Second, an exclusive **prefix sum** is applied to the histogram in order to determine the starting offset of each partition inside the output array. Third, a **scatter step** rearranges the records so that all records belonging to the same partition occupy a contiguous memory region.

This layout is important because the join phase works partition by partition. By storing records of the same partition contiguously, the implementation improves locality of reference and reduces unnecessary memory movement during the join.

In the code, timing is collected not only for the full partitioning step but also for its main sub-phases: histogram, prefix sum, and scatter. This was added to satisfy the requirement that the report should discuss the execution time across the main phases of the algorithm.

Join Phase

After partitioning, the join is executed independently on each partition. For a given partition, the sequential implementation first builds a hash table on the partition of relation R by counting how many times each key appears. Then, the corresponding partition of relation S is scanned. For each key found in S , the algorithm checks whether the same key exists in the hash table built from R . If it does, the number of matches added to the final result is equal to the multiplicity stored for that key in R .

This mechanism is what allows the implementation to support duplicates correctly. The final output consists of the total number of matches (`join_count`) and two checksum values, which are used for correctness verification.

3 Parallel Implementation

The parallel implementation preserves exactly the same global structure as the sequential one: relation R is partitioned, relation S is partitioned, corresponding partitions are joined, and the final result is accumulated. The main design question was therefore not how to change the algorithm, but how to parallelize the expensive phases while keeping the implementation correct and efficient.

Parallelization Strategy

The histogram phase was parallelized because it is naturally data-parallel. Each thread processes a disjoint chunk of the input relation and updates a private local histogram. This avoids synchronization on every update; otherwise, locks or atomic operations would dominate the cost. At the end of the phase, the private histograms are merged into a global histogram.

The prefix sum itself was not aggressively parallelized. In the actual implementation, this phase also includes the merge of local histograms and the computation of per-thread offsets used later in the scatter phase. Experimental results show that this part is extremely small compared to histogram, scatter, and join, so keeping it simple is a reasonable choice.

The scatter phase was parallelized using precomputed thread-specific offsets. Once the offsets are known, each thread writes its own records into a reserved region of the output array. This avoids locks and ensures that the scatter phase remains free of synchronization during the actual writes.

The join phase was parallelized across partitions. After partitioning, partitions are independent, so different threads can process different partition ranges without interference. In each thread, the same build-and-probe logic as in the sequential version is applied locally. This is the phase that benefits most naturally from parallelism because it has both significant work and very low dependence across threads.

Engineering Choices

The implementation reflects a set of deliberate engineering decisions. Expensive and naturally parallel phases, such as histogram, scatter, and join, were parallelized. Lightweight phases, such as prefix sum and offset setup, were kept simple because their measured cost is negligible. The objective was not to maximize the amount of code running in parallel, but to parallelize only where the benefit justifies the additional complexity.

Another important choice was to keep the sequential baseline inside the parallel executable for correctness checks and direct comparison on the same generated input. This makes the validation process straightforward and ensures that the same input data and the same mapping function are

used consistently.

4 Correctness

Correctness is verified using two complementary methods. For small inputs, a naive brute-force join is used as a stronger reference. Since the input is tiny, the $O(N^2)$ cost of the naive method is acceptable and provides a direct way to validate the optimized implementation. In this case, the values of `join_count`, `checksum1`, and `checksum2` are compared against the naive result.

For larger inputs, correctness is verified by comparing the output of the parallel implementation against the sequential reference implementation. In all experiments reported in this project, the values of `join_count`, `checksum1`, and `checksum2` match exactly, and all reported correctness checks pass.

The correctness mechanism is kept outside the measured region. This is important because validation should not distort the timing results, especially when evaluating performance.

5 Performance Evaluation

All performance results reported here were obtained on node09 of the SPM cluster. The raw outputs of all experiments are stored in the `reports/` folder, including `small.txt`, `medium.txt`, `large.txt`, `strong.txt`, `weak.txt`, and `vary_p.txt`. Unless otherwise stated, the speedup values reported in the tables are computed using the sequential baseline shown in the same table.

Small Correctness Test

The small test mainly serves as a correctness validation. It uses $NR = 10$, $NS = 10$, and $P = 4$, and also compares the optimized output against the naive brute-force join.

Version	Time (s)	Speedup
Sequential	0.000008	1.00x
Parallel (4 threads)	0.001405	0.01x

This result is expected: for such a small input, parallelism is not beneficial because thread-management overhead dominates the execution time.

Medium Dataset

For the medium test, $NR = NS = 10^6$ and $P = 128$.

Version	Time (s)	Speedup
Sequential	0.058220	1.00x
Parallel (8 threads)	0.018508	3.15x

At this scale, the workload is large enough for the benefits of parallelism to outweigh the overhead. This is the first experiment where a clear performance advantage appears.

Large Dataset

The main benchmark uses $NR = NS = 10^7$ and $P = 256$.

Version	Time (s)	Speedup
Sequential	0.689216	1.00x
Parallel (8 threads)	0.176708	3.90x

This result confirms that the chosen parallelization strategy is effective on large datasets.

Phase Breakdown

To satisfy the requirement of discussing the main phases of the algorithm, a detailed timing breakdown was added to both the sequential and the parallel implementations. For the large dataset, the measured times are:

Phase	Sequential (s)	Parallel (s)
Histogram R	0.050885	0.007606
Prefix / Offset R	0.000003	0.000011
Scatter R	0.128793	0.057516
Histogram S	0.051817	0.008752
Prefix / Offset S	0.000003	0.000014
Scatter S	0.128689	0.057188
Join	0.328965	0.045603

These measurements show clearly that histogram, scatter, and join all benefit from parallelism, while the prefix / offset phase remains negligible in both versions. This validates the engineering choice of focusing parallelization effort on the expensive phases.

6 Scalability Analysis

Strong Scalability

Strong scalability was evaluated with fixed input size $NR = NS = 2 \times 10^7$ and $P = 256$.

Threads	Parallel Time (s)	Speedup
1	1.356249	1.01x
2	0.790697	1.73x
4	0.488026	2.91x
8	0.343345	3.96x
16	0.314671	4.35x

The execution time decreases as the number of threads increases, which shows good strong scalability. However, the speedup is sub-linear, and the gain from 8 to 16 threads is smaller than the gain from 4 to 8 threads. This behavior is expected in practice and is mainly caused by memory bandwidth limitations, thread overhead, and the presence of small sequential parts.

Weak Scalability

Weak scalability was evaluated by increasing the problem size proportionally to the number of threads, starting from 5×10^6 records per relation per thread.

Threads	Problem Size ($NR = NS$)	Parallel Time (s)
1	5×10^6	0.422203
2	10×10^6	0.404593
4	20×10^6	0.492245
8	40×10^6	0.676724
16	80×10^6	1.233501

Ideally, weak scalability would keep execution time approximately constant, because the work per thread remains constant. In practice, the time increases moderately as the total data size grows. This indicates reasonable weak scalability, but also shows the increasing influence of memory pressure and cache inefficiency at larger scales.

Effect of Partition Count

An additional experiment was performed with fixed $NR = NS = 10^7$ and 8 threads while varying the number of partitions.

P	Sequential (s)	Parallel (s)
64	0.830441	0.205785
128	0.749533	0.190623
256	0.653828	0.176925
512	0.656351	0.204085
1024	0.625535	0.173259

The results show that the partition count has a measurable effect on performance. Increasing P generally improves locality and can reduce the cost of join processing, but the trend is not strictly monotonic because partition management overhead and runtime variability also influence the results. In this experiment, the best parallel time is observed at $P = 1024$, while $P = 256$ also performs strongly. Overall, the results indicate that the choice of P matters, but there is no single universally optimal value.

7 Discussion

The results obtained in this project show that the parallel implementation is both correct and effective. The correctness checks confirm that the sequential and parallel versions always produce identical results, and the small test further confirms this through comparison with a naive join.

From the performance point of view, the results illustrate the typical behavior of a parallel application. For tiny workloads, parallelism is not advantageous because overhead dominates. For medium and large workloads, the computational work becomes large enough that the thread overhead is amortized, and the benefits of parallel execution become clear.

The phase breakdown is particularly useful because it connects the implementation strategy with the measured results. Histogram and scatter phases benefit significantly from parallel execution, while the join phase benefits the most. In contrast, the prefix / offset phase contributes only a negligible fraction of the total time, so keeping it simple is justified both conceptually and experimentally.

The scalability experiments also support the design choices. Strong scalability is good but clearly sub-linear, which is consistent with the memory-bound nature of the algorithm. Weak scalability is reasonable, although execution time still grows with the total amount of data. The main practical limitation is memory bandwidth, since the algorithm performs heavy memory traffic during partitioning and join.

Overall, the experimental results support the adopted parallelization strategy and show that the design choices are reasonable and well justified.

8 Conclusion

This project successfully implements a partitioned hash join with duplicates in both sequential and parallel versions.

The sequential implementation integrates the Module 1 mapping function and serves as the correctness reference and baseline. The parallel implementation preserves the same overall algorithmic structure and achieves a substantial reduction in execution time on medium and large datasets.

The experiments on node09 show that histogram, scatter, and join benefit from parallelism, while prefix / offset computation remains negligible. The performance results, the scalability analysis, and the phase breakdown together demonstrate that careful, phase-aware parallelization provides a clear advantage while also revealing the practical limits imposed by memory bandwidth and parallel overhead.